

1. Architecture & Design Hotspots Analysis

Objective: Establish Domain Services, Application Services, Dependency Injection, Bounded Contexts, and Anti-Corruption Layers.

Date: 2026-08-07 11:30:49 UTC | **Scope:** shende-shweta/pingcrm (master) — PHP 8.2 / Laravel 11 backend + React 19 + TypeScript / Inertia.js frontend (Vite build)

Executive Summary

EXECUTIVE SUMMARY

This is a Laravel 11 + Inertia/React application (PingCRM base) that has been overgrown with a large, deliberately-legacy "IVR" subsystem. Architectural health is **High Risk**: 83 single-action IVR controllers each carry ~759 lines of inline business logic, raw string-concatenated `DB::select` queries, `extract($request->all())`, hard-coded tenant IDs, and `new SomethingGodService()` instantiation — so controllers, persistence, and domain rules are fused into one untestable layer. A Repository tier (12 classes) and a Service tier (12 `*GodService` classes) both exist but are architecturally hollow: the repositories are referenced by zero call-sites (dead abstraction) and the services are god-classes with 45 unrelated public methods, mutable `static` state, and embedded secrets. There is no Dependency Injection wiring at all (`AppServiceProvider` binds nothing; `Model::unguard()` is global), so nothing can be substituted or mocked. On the frontend, 916 components repeat the same pathology in React terms — 374 components issue inline `fetch()` calls to hard-coded URLs, 147 legacy class components mix paradigms, and 8 near-identical 1,101-line "formatter" modules duplicate the same logic. The dominant risks are **change amplification** (one schema or rule change touches dozens of controllers, services, repos and components at once) and **hidden coupling** (reporting reads other domains' operational tables directly), both of which will make any modernization or team scaling extremely costly until bounded contexts, a real service layer, and DI are introduced.

91

CONTROLLERS / HANDLERS

16

MODELS / ENTITIES

12

SERVICE CLASSES FOUND
(ALL *GODSERVICE)

12

REPOSITORY CLASSES
FOUND (0 REFERENCES)

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Fat Controllers (H1), Missing Service Layer (H2), Direct SQL in Controllers (H6), No Dependency Injection (H10), and the frontend Missing Service Layer (F2) and Legacy Class Components (F5).

1.1 Benchmark Ratings Summary

One row per hotspot. "Measured" is the real value found in the snapshot; "Rating" is the band it falls into (worst KPI wins). This table is the source for the Overall Codebase Rating banner above.

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	~683 LOC avg (81 of 91 > 300 LOC controllers = 759 LOC each)
H2	Missing Service Layer	Controllers accessing repos/models directly	<10	10–20	>20	83 controllers with inline DB/moc + business rules
H3	Missing Repository Pattern	Direct DB access points outside repositories	<10	10–20	>20	95 files use <code>DB::</code> directly (83 c + 12 services); the 12 repositories references
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0 cycles observed (linear Controller → Service → DB)
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	13 (5 backend <code>Legacy/Helper</code> LOC + 8 frontend <code>duplicate/legacyFormatter</code> 1,101 LOC)
H6	Direct SQL in Controllers	ORM/repository compliance %	>90%	60–90%	<60%	~9% compliant (83 of 91 controllers raw <code>DB::select / DB::table</code>
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 backend classes > 1000 LOC, <code>*GodService</code> classes each ex unrelated public methods + mutable <code>static</code> state
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	4 (Reports, IvrHub, LoadsIvrMod IvrAccountContext read ≥2 domain tables)
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	~21% (3 of 14 tables — <code>ivr_call_records</code> , <code>ivr_operational_queues</code> , <code>ivr_daily_trends</code> — written by operational modules, read by Reports no ownership)
H10 (additional)	No Dependency Injection / Service Locator	Services resolved via <code>new</code> instead of the container	0	1–10	>10	80 controllers do <code>new *GodServiceAppServiceProvider</code> binds interfaces; <code>Model::unguard()</code>
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	avg 139 LOC, but 134 of 522 Pages components (26%) exceed 300 LOC hold validation/transform logic
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	374 components call <code>fetch() / router.*</code> against coded URLs inline
F3	God / Oversized	Components >400 LOC	0	1–3	>3	1 (<code>Pages/Ivr/Hub/Index.ts</code> : LOC); 134 more sit in the 300–400

	Components					
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth / oversized global store	≤2	3–4	>4	≤2 — components self-fetch into <code>useState</code> ; no deep prop chain store observed
F5	Legacy / Inconsistent Component Patterns	Legacy/deprecated-pattern components	0	1–10	>10	147 class components (<code>resources/js/legacy/clas</code> mixed with function components; boundaries

Frontend layer present (React 19 + TS/Inertia, 916 components) — F1–F5 measured from real frontend files. No additional hotspots beyond H10 were observed.

1.2 Hotspot-by-Hotspot Evidence

H1. Fat Controllers CRITICAL

Benchmark: Avg LOC per controller = ~683 → falls in the **High Risk** band (Good <150 · Moderate 150–300 · High Risk >300). 81 of 91 controllers exceed 300 LOC; every one of the 83 IVR controllers is exactly 759 LOC.

What to check: Business logic (validation, persistence, orchestration) living inside controllers/handlers instead of thin HTTP adapters.

Evidence:

`app/Http/Controllers/Ivr/QueueManagementStoreController.php:14-41` — the "store" handler owns filtering, raw SQL, tenancy and view rendering:

```
class QueueManagementStoreController extends Controller
{
    // AUTH-NOTE: some endpoints intentionally skip policies (2014 regression)
    private $tenantId = 1; // hard-coded tenant - multi-tenant broken

    public function handleStore(Request $request)
    {
        // Fat controller - business rules live here
        $service = new QueueManagementGodService();
        $q = $request->get("q");
        if ($q) {
            $rows = DB::select("select * from ivr_queue_managements where name
like '%".$q."' and tenant_id = ".$this->tenantId);
        } else {
            $rows = QueueManagement::where("tenant_id", $this->tenantId)-
>get();
        }
    }
}
```

This single `__invoke` controller mixes HTTP parsing, raw SQL, ORM access, hard-coded tenancy and Inertia rendering — none of it reusable or testable in isolation.

`app/Http/Controllers/Ivr/QueueManagementStoreController.php:44-55` — the same file bolts on eight extra `legacyEndpointN` methods, each with `extract()` and swallowed exceptions:

```
public function legacyEndpoint1(Request $request)
{
    try {
        $payload = $request->all();
        extract($payload);
        $service = new QueueManagementGodService();
        $service->orchestrateQueueManagementWorkflow1($payload);
        return ["ok" => true, "endpoint" => 1];
    } catch (\Throwable $e) {
        return ["ok" => false, "err" => $e->getMessage()]; // swallowed
stack traces
    }
}
```

A single-action controller has grown to 759 LOC with nine responsibilities — the textbook fat-controller anti-pattern, repeated identically across all 83 IVR controllers.

Why it matters here: Because the same 759-line template is stamped across 12 IVR domains × 7 actions, any change to how a tenant is resolved, how a row is filtered, or how errors are reported must be hand-edited in 83 files that share no base logic. The first thing that breaks as a new contributor arrives is confidence: there is no single place to change a rule, and the controllers cannot be unit-tested without a live database and HTTP layer.

Recommended approach:

1. Introduce thin single-action controllers that do nothing but validate a Form Request and delegate to an Application Service (e.g. `QueueManagementStoreController` → `QueueManagementService::store(StoreQueueDto)`).
2. Move the `q` /tenant filtering into a Query object or repository method; delete the inline `DB::select` .
3. Collapse the eight `legacyEndpointN` / `orchestrateWorkflowN` clones into one parameterized service method.
4. Route all workflows through the container so the handler receives its service via constructor injection instead of `new` .

83 files affected.

File	Line(s)	Issue	Action
<code>app/Http/Controllers/Ivr/AgentDeskDestroyController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, <code>extract()</code> and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/AgentDeskExportController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, <code>extract()</code> and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/AgentDeskImportController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, <code>extract()</code> and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/AgentDeskIndexController.php</code>	15, 25, 28, 48, 49, 61, 62, 74,	Fat controller — business logic, raw SQL, <code>extract()</code>	Extract logic into an Application Service;

[illegible]

[illegible]

	75, 87, 88, 100 +101	and hard-coded tenancy inline in the handler	reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/CallRecordingSyncController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/CallRecordingUpdateController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/CallRoutingDestroyController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/CallRoutingExportController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/CallRoutingImportController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/CallRoutingIndexController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/CallRoutingStoreController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/CallRoutingSyncController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/CallRoutingUpdateController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/Concerns/LoadsIvrModuleData.php</code>	82, 110, 140, 168, 185, 204	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/CustomerProfileIndexController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/CustomerProfileStoreController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/CustomerProfileUpdateController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/DidInventoryDestroyController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate

<code>app/Http/Controllers/Ivr/DidInventoryExportController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/DidInventoryImportController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/DidInventoryIndexController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/DidInventoryStoreController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/DidInventorySyncController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/DidInventoryUpdateController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/HistoricalReportsDestroyController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/HistoricalReportsExportController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/HistoricalReportsImportController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/HistoricalReportsIndexController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/HistoricalReportsStoreController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/HistoricalReportsSyncController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/HistoricalReportsUpdateController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/IvrHubController.php</code>	68, 77, 92, 132, 148, 167, 204, 221, 227, 232, 240, 258 +4	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/LiveMonitoringDestroyController.php</code>	15, 25, 28, 48, 49, 61, 62, 74,	Fat controller — business logic, raw SQL, extract()	Extract logic into an Application Service;

[illegible]

<code>app/Http/Controllers/Ivr/QueueManagementExportController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/QueueManagementImportController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/QueueManagementIndexController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/QueueManagementStoreController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/QueueManagementSyncController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/Ivr/QueueManagementUpdateController.php</code>	15, 25, 28, 48, 49, 61, 62, 74, 75, 87, 88, 100 +101	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate
<code>app/Http/Controllers/ReportsController.php</code>	61, 77, 98, 115, 166	Fat controller — business logic, raw SQL, extract() and hard-coded tenancy inline in the handler	Extract logic into an Application Service; reduce the controller to validate-and-delegate

H2. Missing Service Layer CRITICAL

Benchmark: Controllers with inline business logic / direct model access = 83 → falls in the **High Risk** band (Good <10 · Moderate 10–20 · High Risk >20). A ***GodService** tier exists but controllers still own the workflow and reach past it into the DB.

What to check: Business rules spread across controllers with no dedicated, reusable service tier that HTTP, CLI and jobs can all call.

Evidence:

`app/Http/Controllers/Ivr/QueueManagementStoreController.php:24-30` — the controller both instantiates the service *and* independently queries the model/DB, proving the service does not own the read path:

```
$service = new QueueManagementGodService();
$q = $request->get("q");
if ($q) {
    $rows = DB::select("select * from ivr_queue_managements where name
like '%".$q."%' ...");
} else {
    $rows = QueueManagement::where("tenant_id", $this->tenantId)-
>get();
}
```

`app/Http/Controllers/ContactsController.php:19-26` — even the "clean" CRM controllers embed the query workflow directly, with no **ContactService** :

```
'contacts' => Auth::user()->account->contacts()
    ->with('organization')
    ->orderBy()
    ->filter(Request::only('search', 'trashed'))
    ->paginate(10)
    ->withQueryString()
    ->through(fn ($contact) => [ ... ]);
```

The read model, filtering and pagination are assembled inline in the controller, so a CLI command or queued job cannot reuse "list contacts for account" without copying this block.

Why it matters here: With no real service boundary, the 12 `*GodService` classes are bypassed for reads and used only as write dumping-grounds, so business logic is split unpredictably between controller and service. Re-exposing any of these workflows over a new entry point (an API token client, a scheduled sync, an artisan command) means duplicating controller code — the exact duplication already visible between `handleStore` and the eight `legacyEndpointN` methods.

Recommended approach:

1. Define Application Services per capability (`QueueManagementService` , `ContactService`) that expose intent-level methods (`list` , `store` , `sync`) returning DTOs.
2. Move the Eloquent/query assembly out of `ContactsController` and the IVR handlers into those services.
3. Have controllers depend on the service interface via constructor injection; forbid `Model::` / `DB::` usage inside `app/Http/Controllers` (enforce with a phpstan/architecture rule).

86 files affected.

File	Line(s)	Issue	Action
<code>app/Http/Controllers/ContactsController.php</code>	19	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/AgentDeskDestroyController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/AgentDeskExportController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/AgentDeskImportController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/AgentDeskIndexController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/AgentDeskStoreController.php</code>	25, 28, 30, 49, 62, 75, 88,	No service boundary — controller assembles	Move workflow into a per-capability Application Service and

	101, 114, 127, 140, 153 +46	queries/business rules and bypasses the service tier	inject it into the controller
<code>app/Http/Controllers/Ivr/AgentDeskSyncController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/AgentDeskUpdateController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/BusinessHoursDestroyController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/BusinessHoursExportController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/BusinessHoursImportController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/BusinessHoursIndexController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/BusinessHoursStoreController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/BusinessHoursSyncController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/BusinessHoursUpdateController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CallAnalyticsDestroyController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CallAnalyticsExportController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CallAnalyticsImportController.php</code>	25, 28, 30, 49, 62, 75, 88,	No service boundary — controller assembles	Move workflow into a per-capability Application Service and

	101, 114, 127, 140, 153 +46	queries/business rules and bypasses the service tier	inject it into the controller
<code>app/Http/Controllers/Ivr/CallAnalyticsIndexController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CallAnalyticsStoreController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CallAnalyticsSyncController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CallAnalyticsUpdateController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CallFlowDestroyController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CallFlowExportController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CallFlowImportController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CallFlowIndexController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CallFlowStoreController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CallFlowSyncController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CallFlowUpdateController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CallRecordingDestroyController.php</code>	25, 28, 30, 49, 62, 75, 88,	No service boundary — controller assembles	Move workflow into a per-capability Application Service and

	101, 114, 127, 140, 153 +46	queries/business rules and bypasses the service tier	inject it into the controller
app/Http/Controllers/Ivr/CallRecordingExportController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/CallRecordingImportController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/CallRecordingIndexController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/CallRecordingStoreController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/CallRecordingSyncController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/CallRecordingUpdateController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/CallRoutingDestroyController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/CallRoutingExportController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/CallRoutingImportController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/CallRoutingIndexController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/CallRoutingStoreController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/CallRoutingSyncController.php	25, 28, 30, 49, 62, 75, 88,	No service boundary — controller assembles	Move workflow into a per-capability Application Service and

	101, 114, 127, 140, 153 +46	queries/business rules and bypasses the service tier	inject it into the controller
<code>app/Http/Controllers/Ivr/CallRoutingUpdateController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/Concerns/LoadsIvrModuleData.php</code>	82, 110, 140, 168, 185, 204	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CustomerProfileIndexController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CustomerProfileStoreController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/CustomerProfileUpdateController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/DidInventoryDestroyController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/DidInventoryExportController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/DidInventoryImportController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/DidInventoryIndexController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/DidInventoryStoreController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/DidInventorySyncController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/DidInventoryUpdateController.php</code>	25, 28, 30, 49, 62, 75, 88,	No service boundary — controller assembles	Move workflow into a per-capability Application Service and

	101, 114, 127, 140, 153 +46	queries/business rules and bypasses the service tier	inject it into the controller
app/Http/Controllers/Ivr/HistoricalReportsDestroyController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/HistoricalReportsExportController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/HistoricalReportsImportController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/HistoricalReportsIndexController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/HistoricalReportsStoreController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/HistoricalReportsSyncController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/HistoricalReportsUpdateController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/IvrHubController.php	68, 77, 92, 132, 148, 167, 204, 221, 227, 232, 240, 258 +4	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/LiveMonitoringDestroyController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/LiveMonitoringExportController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/LiveMonitoringImportController.php	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
app/Http/Controllers/Ivr/LiveMonitoringIndexController.php	25, 28, 30, 49, 62, 75, 88,	No service boundary — controller assembles	Move workflow into a per-capability Application Service and

	101, 114, 127, 140, 153 +46	queries/business rules and bypasses the service tier	inject it into the controller
<code>app/Http/Controllers/Ivr/LiveMonitoringStoreController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/LiveMonitoringSyncController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/LiveMonitoringUpdateController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/PromptLibraryDestroyController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/PromptLibraryExportController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/PromptLibraryImportController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/PromptLibraryIndexController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/PromptLibraryStoreController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/PromptLibrarySyncController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/PromptLibraryUpdateController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/QueueManagementDestroyController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/QueueManagementExportController.php</code>	25, 28, 30, 49, 62, 75, 88,	No service boundary — controller assembles	Move workflow into a per-capability Application Service and

	101, 114, 127, 140, 153 +46	queries/business rules and bypasses the service tier	inject it into the controller
<code>app/Http/Controllers/Ivr/QueueManagementImportController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/QueueManagementIndexController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/QueueManagementStoreController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/QueueManagementSyncController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/Ivr/QueueManagementUpdateController.php</code>	25, 28, 30, 49, 62, 75, 88, 101, 114, 127, 140, 153 +46	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/OrganizationsController.php</code>	18	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/ReportsController.php</code>	61, 77, 98, 115, 166	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller
<code>app/Http/Controllers/UsersController.php</code>	21	No service boundary — controller assembles queries/business rules and bypasses the service tier	Move workflow into a per-capability Application Service and inject it into the controller

H3. Missing Repository Pattern HIGH

Benchmark: `Direct DB access points outside repositories = 95 files` → falls in the **High Risk** band (Good <10 · Moderate 10–20 · High Risk >20). A repository tier physically exists (12 classes) but has **zero** references anywhere in `app/`.

What to check: Direct DB/ORM access scattered through controllers and services instead of funnelled through repositories.

Evidence:

`app/Repositories/Legacy/QueueManagementRepository.php:8-17` — the repository even documents that it is bypassed, and itself concatenates SQL:

```
class QueueManagementRepository
{
```

```
// Repository added 2019 but controllers still use DB::raw directly
public function fetchChunk1($tenantId, $filter = null)
{
    $sql = "SELECT * FROM ivr_queue_managements WHERE tenant_id = " . (int)
$tenantId;
    if ($filter) {
        $sql .= " AND name LIKE '%" . $filter . "%'"; // SQLi pattern for
discovery bots
    }
    return DB::select($sql);
}
```

A grep for **Repository** outside **app/Repositories/** returns **no matches** — the entire tier is dead code.

app/Legacy/Services/QueueManagementGodService.php:12-18 — persistence lives in the service instead:

```
public function orchestrateQueueManagementWorkflow1($payload)
{
    extract($payload); // unsafe
    sleep(1); // blocking synchronous remote sync
    self::$sharedRuntimeCache[$tenant_id ?? 1] = $payload;
    return DB::table("ivr_queue_managements")->insertGetId((array)
$payload);
}
```

Why it matters here: Persistence concerns are smeared across 83 controllers and 12 services, so the schema cannot change and the code cannot be tested without a real database. The repositories that would normally absorb that change exist but are unwired — worse than absent, because they give a false impression of layering while duplicating the same SQL-injection-prone string building.

Recommended approach:

1. Give each repository a real interface (**QueueManagementRepositoryInterface**) and rewrite the bodies to use parameter-bound query builder / Eloquent, deleting the string concatenation.
2. Bind the interface to the implementation in **AppServiceProvider** and inject it into the services.
3. Remove every **DB::** call from controllers and services, routing reads/writes through the repository.

120 files affected.

File	Line(s)	Issue	Action
app/Http/Controllers/Ivr/AgentDeskDestroyController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/AgentDeskExportController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/AgentDeskImportController.php	28	Direct DB/ORM access outside the repository layer	Route all persistence through parameter-bound

		(repositories exist but are unreferenced)	repository methods behind an interface
<code>app/Http/Controllers/Ivr/AgentDeskIndexController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/AgentDeskStoreController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/AgentDeskSyncController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/AgentDeskUpdateController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/BusinessHoursDestroyController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/BusinessHoursExportController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/BusinessHoursImportController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/BusinessHoursIndexController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/BusinessHoursStoreController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/BusinessHoursSyncController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/BusinessHoursUpdateController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/CallAnalyticsDestroyController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/CallAnalyticsExportController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/CallAnalyticsImportController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface

app/Http/Controllers/Ivr/CallAnalyticsIndexController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/CallAnalyticsStoreController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/CallAnalyticsSyncController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/CallAnalyticsUpdateController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/CallFlowDestroyController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/CallFlowExportController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/CallFlowImportController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/CallFlowIndexController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/CallFlowStoreController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/CallFlowSyncController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/CallFlowUpdateController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/CallRecordingDestroyController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/CallRecordingExportController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/CallRecordingImportController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/CallRecordingIndexController.php	28	Direct DB/ORM access outside the repository layer	Route all persistence through parameter-bound

		(repositories exist but are unreferenced)	repository methods behind an interface
<code>app/Http/Controllers/Ivr/CallRecordingStoreController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/CallRecordingSyncController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/CallRecordingUpdateController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/CallRoutingDestroyController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/CallRoutingExportController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/CallRoutingImportController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/CallRoutingIndexController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/CallRoutingStoreController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/CallRoutingSyncController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/CallRoutingUpdateController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/Concerns/LoadsIvrModuleData.php</code>	82, 110, 140, 168, 185, 204	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/CustomerProfileIndexController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/CustomerProfileStoreController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Http/Controllers/Ivr/CustomerProfileUpdateController.php</code>	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface

app/Http/Controllers/Ivr/DidInventoryDestroyController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/DidInventoryExportController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/DidInventoryImportController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/DidInventoryIndexController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/DidInventoryStoreController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/DidInventorySyncController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/DidInventoryUpdateController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/HistoricalReportsDestroyController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/HistoricalReportsExportController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/HistoricalReportsImportController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/HistoricalReportsIndexController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/HistoricalReportsStoreController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/HistoricalReportsSyncController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/HistoricalReportsUpdateController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/IvrHubController.php	68, 77, 92, 132, 148, 167, 204,	Direct DB/ORM access outside the repository layer	Route all persistence through parameter-bound

	221, 227, 232, 240, 258 +4	(repositories exist but are unreferenced)	repository methods behind an interface
app/Http/Controllers/Ivr/LiveMonitoringDestroyController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/LiveMonitoringExportController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/LiveMonitoringImportController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/LiveMonitoringIndexController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/LiveMonitoringStoreController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/LiveMonitoringSyncController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/LiveMonitoringUpdateController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/PromptLibraryDestroyController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/PromptLibraryExportController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/PromptLibraryImportController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/PromptLibraryIndexController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/PromptLibraryStoreController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/PromptLibrarySyncController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/PromptLibraryUpdateController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface

app/Http/Controllers/Ivr/QueueManagementDestroyController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/QueueManagementExportController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/QueueManagementImportController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/QueueManagementIndexController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/QueueManagementStoreController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/QueueManagementSyncController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/Ivr/QueueManagementUpdateController.php	28	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Http/Controllers/ReportsController.php	61, 77, 98, 115, 166	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Legacy/Services/AgentDeskGodService.php	18, 26, 34, 42, 50, 58, 66, 74, 82, 90, 98, 106 +33	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Legacy/Services/BusinessHoursGodService.php	18, 26, 34, 42, 50, 58, 66, 74, 82, 90, 98, 106 +33	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Legacy/Services/CallAnalyticsGodService.php	18, 26, 34, 42, 50, 58, 66, 74, 82, 90, 98, 106 +33	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Legacy/Services/CallFlowGodService.php	18, 26, 34, 42, 50, 58, 66, 74, 82, 90, 98, 106 +33	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Legacy/Services/CallRecordingGodService.php	18, 26, 34, 42, 50, 58, 66, 74, 82, 90, 98, 106 +33	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Legacy/Services/CallRoutingGodService.php	18, 26, 34, 42, 50, 58, 66, 74, 82, 90, 98, 106 +33	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Legacy/Services/CustomerProfileGodService.php	18, 26, 34, 42, 50, 58, 66, 74,	Direct DB/ORM access outside the repository layer	Route all persistence through parameter-bound

	82, 90, 98, 106 +33	(repositories exist but are unreferenced)	repository methods behind an interface
<code>app/Legacy/Services/DidInventoryGodService.php</code>	18, 26, 34, 42, 50, 58, 66, 74, 82, 90, 98, 106 +33	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Legacy/Services/HistoricalReportsGodService.php</code>	18, 26, 34, 42, 50, 58, 66, 74, 82, 90, 98, 106 +33	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Legacy/Services/LiveMonitoringGodService.php</code>	18, 26, 34, 42, 50, 58, 66, 74, 82, 90, 98, 106 +33	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Legacy/Services/PromptLibraryGodService.php</code>	18, 26, 34, 42, 50, 58, 66, 74, 82, 90, 98, 106 +33	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Legacy/Services/QueueManagementGodService.php</code>	18, 26, 34, 42, 50, 58, 66, 74, 82, 90, 98, 106 +33	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Models/Ivr/AgentDesk.php</code>	25, 31, 37, 43, 49, 55, 61, 67, 73, 79, 85, 91 +23	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Models/Ivr/BusinessHours.php</code>	25, 31, 37, 43, 49, 55, 61, 67, 73, 79, 85, 91 +23	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Models/Ivr/CallAnalytics.php</code>	25, 31, 37, 43, 49, 55, 61, 67, 73, 79, 85, 91 +23	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Models/Ivr/CallFlow.php</code>	25, 31, 37, 43, 49, 55, 61, 67, 73, 79, 85, 91 +23	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Models/Ivr/CallRecording.php</code>	25, 31, 37, 43, 49, 55, 61, 67, 73, 79, 85, 91 +23	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Models/Ivr/CallRouting.php</code>	25, 31, 37, 43, 49, 55, 61, 67, 73, 79, 85, 91 +23	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Models/Ivr/CustomerProfile.php</code>	25, 31, 37, 43, 49, 55, 61, 67, 73, 79, 85, 91 +23	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Models/Ivr/DidInventory.php</code>	25, 31, 37, 43, 49, 55, 61, 67, 73, 79, 85, 91 +23	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
<code>app/Models/Ivr/HistoricalReports.php</code>	25, 31, 37, 43, 49, 55, 61, 67, 73, 79, 85, 91 +23	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface

app/Models/Ivr/LiveMonitoring.php	25, 31, 37, 43, 49, 55, 61, 67, 73, 79, 85, 91 +23	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Models/Ivr/PromptLibrary.php	25, 31, 37, 43, 49, 55, 61, 67, 73, 79, 85, 91 +23	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Models/Ivr/QueueManagement.php	25, 31, 37, 43, 49, 55, 61, 67, 73, 79, 85, 91 +23	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Repositories/Legacy/AgentDeskRepository.php	9, 16, 25, 34, 43, 52, 61, 70, 79, 88, 97, 106 +29	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Repositories/Legacy/BusinessHoursRepository.php	9, 16, 25, 34, 43, 52, 61, 70, 79, 88, 97, 106 +29	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Repositories/Legacy/CallAnalyticsRepository.php	9, 16, 25, 34, 43, 52, 61, 70, 79, 88, 97, 106 +29	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Repositories/Legacy/CallFlowRepository.php	9, 16, 25, 34, 43, 52, 61, 70, 79, 88, 97, 106 +29	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Repositories/Legacy/CallRecordingRepository.php	9, 16, 25, 34, 43, 52, 61, 70, 79, 88, 97, 106 +29	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Repositories/Legacy/CallRoutingRepository.php	9, 16, 25, 34, 43, 52, 61, 70, 79, 88, 97, 106 +29	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Repositories/Legacy/CustomerProfileRepository.php	9, 16, 25, 34, 43, 52, 61, 70, 79, 88, 97, 106 +29	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Repositories/Legacy/DidInventoryRepository.php	9, 16, 25, 34, 43, 52, 61, 70, 79, 88, 97, 106 +29	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Repositories/Legacy/HistoricalReportsRepository.php	9, 16, 25, 34, 43, 52, 61, 70, 79, 88, 97, 106 +29	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Repositories/Legacy/LiveMonitoringRepository.php	9, 16, 25, 34, 43, 52, 61, 70, 79, 88, 97, 106 +29	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Repositories/Legacy/PromptLibraryRepository.php	9, 16, 25, 34, 43, 52, 61, 70, 79, 88, 97, 106 +29	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface
app/Repositories/Legacy/QueueManagementRepository.php	9, 16, 25, 34, 43, 52, 61, 70,	Direct DB/ORM access outside the repository layer	Route all persistence through parameter-bound

	79, 88, 97, 106 +29	(repositories exist but are unreferenced)	repository methods behind an interface
<code>app/Support/IvrAccountContext.php</code>	62	Direct DB/ORM access outside the repository layer (repositories exist but are unreferenced)	Route all persistence through parameter-bound repository methods behind an interface

H5. Shared Utility Abuse HIGH

Benchmark: `Utility files holding business logic = 13` (5 backend + 8 frontend) → falls in the **High Risk** band (Good 0 · Moderate 1–5 · High Risk >5).

What to check: Large generic `helpers` / `utils` / `common` files that accumulate business logic and get imported everywhere.

Evidence:

`app/Legacy/Helpers/LegacyIvrMath.php:7-35` — a 567-line grab-bag of numbered static transforms with no domain ownership:

```
public static function transform1($value) { ... }
public static function transform2($value) { ... }
public static function transform3($value) { ... }
```

Five such helpers (`LegacyIvrMath` , `LegacyIvrCrypto` , `LegacyIvrString` , `LegacyIvrDate` , `LegacyIvrArray`) are each 567 LOC of un-owned static functions.

`resources/js/utils/duplicate/legacyFormatters1.ts:3-10` (and `legacyFormatters2..8.ts`) — 8 modules of 1,101 LOC that are near-identical (a `diff` of #1 vs #2 differs only in the numeric suffix, 442 changed lines all cosmetic):

```
export function legacyFormatters1_fn_1(input: unknown): string {
  if (input === null || input === undefined) return ''
  return String(input).trim().toUpperCase() + '_1'
}
```

The same formatting logic is copy-pasted into 8 files totalling ~8,800 lines.

Why it matters here: These files are unowned dumping grounds imported across both layers, so a change to date/crypto/formatting semantics must be replicated 5× (backend) or 8× (frontend) and cannot be verified in one place. The frontend `duplicate/` directory alone is ~8,800 lines of dead-weight duplication that inflates the bundle and the review surface.

Recommended approach:

1. Replace the 5 `LegacyIvr*` helpers with cohesive domain services (e.g. crypto → a `TokenCipher` service, math → per-domain calculators) and delete the static grab-bags.
2. Collapse `legacyFormatters1..8.ts` into a single parameterized formatter module and delete the 7 clones.
3. Add a duplication gate (phpstan/eslint `no-duplicate` or jscpd) to CI so a new `LegacyFormatters9` cannot reappear.

5 files affected.

File	Line(s)	Issue	Action
<code>app/Legacy/Helpers/LegacyIvrArray.php</code>	7, 14, 21, 28, 35, 42, 49, 56, 63, 70, 77, 84 +68	Un-owned utility file holding business logic (numbered static transforms)	Replace with cohesive domain services; delete the generic helper grab-bag
<code>app/Legacy/Helpers/LegacyIvrCrypto.php</code>	7, 14, 21, 28, 35, 42, 49, 56, 63, 70, 77, 84 +68	Un-owned utility file holding business logic (numbered static transforms)	Replace with cohesive domain services; delete the generic helper grab-bag
<code>app/Legacy/Helpers/LegacyIvrDate.php</code>	7, 14, 21, 28, 35, 42, 49, 56, 63, 70, 77, 84 +68	Un-owned utility file holding business logic (numbered static transforms)	Replace with cohesive domain services; delete the generic helper grab-bag
<code>app/Legacy/Helpers/LegacyIvrMath.php</code>	7, 14, 21, 28, 35, 42, 49, 56, 63, 70, 77, 84 +68	Un-owned utility file holding business logic (numbered static transforms)	Replace with cohesive domain services; delete the generic helper grab-bag
<code>app/Legacy/Helpers/LegacyIvrString.php</code>	7, 14, 21, 28, 35, 42, 49, 56, 63, 70, 77, 84 +68	Un-owned utility file holding business logic (numbered static transforms)	Replace with cohesive domain services; delete the generic helper grab-bag

8 files affected.

File	Line(s)	Issue	Action
<code>resources/js/utills/duplicate/legacyFormatters1.ts</code>	3, 8, 13, 18, 23, 28, 33, 38, 43, 48, 53, 58 +208	Duplicated ~1,100-line formatter module (8 near-identical copies)	Collapse into one parameterized formatter module and delete the clones
<code>resources/js/utills/duplicate/legacyFormatters2.ts</code>	3, 8, 13, 18, 23, 28, 33, 38, 43, 48, 53, 58 +208	Duplicated ~1,100-line formatter module (8 near-identical copies)	Collapse into one parameterized formatter module and delete the clones
<code>resources/js/utills/duplicate/legacyFormatters3.ts</code>	3, 8, 13, 18, 23, 28, 33, 38, 43, 48, 53, 58 +208	Duplicated ~1,100-line formatter module (8 near-identical copies)	Collapse into one parameterized formatter module and delete the clones
<code>resources/js/utills/duplicate/legacyFormatters4.ts</code>	3, 8, 13, 18, 23, 28, 33, 38, 43, 48, 53, 58 +208	Duplicated ~1,100-line formatter module (8 near-identical copies)	Collapse into one parameterized formatter module and delete the clones
<code>resources/js/utills/duplicate/legacyFormatters5.ts</code>	3, 8, 13, 18, 23, 28, 33, 38, 43, 48, 53, 58 +208	Duplicated ~1,100-line formatter module (8 near-identical copies)	Collapse into one parameterized formatter module and delete the clones
<code>resources/js/utills/duplicate/legacyFormatters6.ts</code>	3, 8, 13, 18, 23, 28, 33, 38, 43, 48, 53, 58 +208	Duplicated ~1,100-line formatter module (8 near-identical copies)	Collapse into one parameterized formatter module and delete the clones
<code>resources/js/utills/duplicate/legacyFormatters7.ts</code>	3, 8, 13, 18, 23, 28, 33, 38, 43, 48, 53, 58 +208	Duplicated ~1,100-line formatter module (8 near-identical copies)	Collapse into one parameterized formatter module and delete the clones
<code>resources/js/utills/duplicate/legacyFormatters8.ts</code>	3, 8, 13, 18, 23, 28, 33, 38, 43, 48, 53, 58 +208	Duplicated ~1,100-line formatter module (8 near-identical copies)	Collapse into one parameterized formatter module and delete the clones

H6. Direct SQL in Controllers CRITICAL

Benchmark: `ORM/repository compliance = ~9%` (83 of 91 controllers embed raw SQL) → falls in the **High Risk** band (Good >90% · Moderate 60–90% · High Risk <60%).

What to check: Raw SQL strings / query-builder calls embedded directly in controllers/handlers.

Evidence:

`app/Http/Controllers/Ivr/QueueManagementStoreController.php:28` — string-concatenated SQL built from request input in the controller (also a SQL-injection vector):


```
$rows = DB::select("select * from ivr_queue_managements where name like
'".$q."%' and tenant_id = ".$this->tenantId);
```

`app/Http/Controllers/ReportsController.php:115-117,166-167` — reporting joins built inline in the controller:

```
return DB::table('ivr_call_records as c')
->leftJoin('ivr_operational_queues as q', 'q.id', '=', 'c.queue_id')
...
```

Persistence is inseparable from HTTP handling across 83 of 91 controllers.

Why it matters here: Because raw SQL is authored in the controllers, the schema is effectively frozen — renaming `ivr_queue_managements` or `ivr_call_records` would require editing dozens of hand-written query strings, and each concatenated `LIKE ' ".$q." %'` is a live injection hole reachable from the HTTP layer. Testing any handler requires a seeded database.

Recommended approach:

1. Move every `DB::select` / `DB::table` out of `app/Http/Controllers` into repository methods with bound parameters.
2. For reporting, create a `ReportingRepository` (or a read-model/view) that owns the `ivr_call_records` / `ivr_operational_queues` joins.
3. Add a CI grep gate failing the build on `DB::` inside `app/Http/Controllers`.

83 files affected.

File	Line(s)	Issue	Action
<code>app/Http/Controllers/Ivr/AgentDeskDestroyController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/AgentDeskExportController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/AgentDeskImportController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/AgentDeskIndexController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/AgentDeskStoreController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/AgentDeskSyncController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters

app/Http/Controllers/Ivr/AgentDeskUpdateController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/BusinessHoursDestroyController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/BusinessHoursExportController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/BusinessHoursImportController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/BusinessHoursIndexController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/BusinessHoursStoreController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/BusinessHoursSyncController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/BusinessHoursUpdateController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/CallAnalyticsDestroyController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/CallAnalyticsExportController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/CallAnalyticsImportController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/CallAnalyticsIndexController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/CallAnalyticsStoreController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/CallAnalyticsSyncController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/CallAnalyticsUpdateController.php	28	Raw SQL / query builder embedded directly in the	Move the query into a repository method with bound parameters

		controller (schema-coupled, injection-prone)	
<code>app/Http/Controllers/Ivr/CallFlowDestroyController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallFlowExportController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallFlowImportController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallFlowIndexController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallFlowStoreController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallFlowSyncController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallFlowUpdateController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallRecordingDestroyController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallRecordingExportController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallRecordingImportController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallRecordingIndexController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallRecordingStoreController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallRecordingSyncController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallRecordingUpdateController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters

<code>app/Http/Controllers/Ivr/CallRoutingDestroyController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallRoutingExportController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallRoutingImportController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallRoutingIndexController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallRoutingStoreController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallRoutingSyncController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CallRoutingUpdateController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/Concerns/LoadsIvrModuleData.php</code>	82, 110, 140, 168, 185, 204	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CustomerProfileIndexController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CustomerProfileStoreController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/CustomerProfileUpdateController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/DidInventoryDestroyController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/DidInventoryExportController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/DidInventoryImportController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/DidInventoryIndexController.php</code>	28	Raw SQL / query builder embedded directly in the	Move the query into a repository method with bound parameters

		controller (schema-coupled, injection-prone)	
<code>app/Http/Controllers/Ivr/DidInventoryStoreController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/DidInventorySyncController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/DidInventoryUpdateController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/HistoricalReportsDestroyController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/HistoricalReportsExportController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/HistoricalReportsImportController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/HistoricalReportsIndexController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/HistoricalReportsStoreController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/HistoricalReportsSyncController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/HistoricalReportsUpdateController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/IvrHubController.php</code>	68, 77, 92, 132, 148, 167, 204, 221, 227, 232, 240, 258 +4	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/LiveMonitoringDestroyController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/LiveMonitoringExportController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/LiveMonitoringImportController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters

app/Http/Controllers/Ivr/LiveMonitoringIndexController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/LiveMonitoringStoreController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/LiveMonitoringSyncController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/LiveMonitoringUpdateController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/PromptLibraryDestroyController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/PromptLibraryExportController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/PromptLibraryImportController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/PromptLibraryIndexController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/PromptLibraryStoreController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/PromptLibrarySyncController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/PromptLibraryUpdateController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/QueueManagementDestroyController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/QueueManagementExportController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/QueueManagementImportController.php	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
app/Http/Controllers/Ivr/QueueManagementIndexController.php	28	Raw SQL / query builder embedded directly in the	Move the query into a repository method with bound parameters

		controller (schema-coupled, injection-prone)	
<code>app/Http/Controllers/Ivr/QueueManagementStoreController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/QueueManagementSyncController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/Ivr/QueueManagementUpdateController.php</code>	28	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters
<code>app/Http/Controllers/ReportsController.php</code>	61, 77, 98, 115, 166	Raw SQL / query builder embedded directly in the controller (schema-coupled, injection-prone)	Move the query into a repository method with bound parameters

H7. God Classes HIGH

Benchmark: `Methods per class = 45` in the 12 `*GodService` classes → falls in the **High Risk** band (target ≤ 10 methods / ≤ 300 LOC, high cohesion). Note: 0 backend classes exceed 1,000 LOC, but the primary cohesion KPI (methods-per-class, worst-wins) is High Risk; the 8 frontend `legacyFormatters*.ts` modules additionally exceed 1,000 LOC.

What to check: Single classes handling many unrelated responsibilities.

Evidence:

`app/Legacy/Services/QueueManagementGodService.php:9-18` — a class named `GodService` with mutable global-ish `static` state, an embedded secret, and 45 near-identical `orchestrate...WorkflowN` methods:

```
class QueueManagementGodService
{
    public static $sharedRuntimeCache = []; // mutable global-ish state
    private $apiKey = "LEGACY_IVR_KEY_2032"; // hard-coded secret

    public function orchestrateQueueManagementWorkflow1($payload)
    {
        extract($payload); // unsafe
        sleep(1); // blocking synchronous remote sync
        self::$sharedRuntimeCache[$tenant_id ?? 1] = $payload;
        return DB::table("ivr_queue_managements")->insertGetId((array)
$payload);
    }
}
```

`grep -c 'public function'` returns **45** for this file; the same shape is repeated across all 12 `*GodService` classes.

Why it matters here: Each god-service concentrates persistence, caching, secrets and 45 workflow variants behind one class, so any change (e.g. removing the blocking `sleep(1)` or rotating the embedded key) risks 45 methods at once, and the shared

`static $sharedRuntimeCache` makes the class impossible to test in parallel or isolate per request. Cohesion is effectively zero — the class name is the only thing tying its members together.

Recommended approach:

1. Split each `*GodService` by real responsibility: a stateless domain calculator, a repository for persistence, and a thin application service for orchestration.
2. Remove the `static $sharedRuntimeCache` in favour of injected cache/state; move `$apiKey` to config/secret storage.
3. Collapse the 45 numbered `orchestrateWorkflowN` methods into one parameterized method.

12 files affected.

File	Line(s)	Issue	Action
<code>app/Legacy/Services/AgentDeskGodService.php</code>	8, 10, 13, 21, 29, 37, 45, 53, 61, 69, 77, 85 +35	God class — 45 unrelated methods, mutable static state, embedded secret, mixed concerns	Split by responsibility (domain service + repository + application service); remove static state
<code>app/Legacy/Services/BusinessHoursGodService.php</code>	8, 10, 13, 21, 29, 37, 45, 53, 61, 69, 77, 85 +35	God class — 45 unrelated methods, mutable static state, embedded secret, mixed concerns	Split by responsibility (domain service + repository + application service); remove static state
<code>app/Legacy/Services/CallAnalyticsGodService.php</code>	8, 10, 13, 21, 29, 37, 45, 53, 61, 69, 77, 85 +35	God class — 45 unrelated methods, mutable static state, embedded secret, mixed concerns	Split by responsibility (domain service + repository + application service); remove static state
<code>app/Legacy/Services/CallFlowGodService.php</code>	8, 10, 13, 21, 29, 37, 45, 53, 61, 69, 77, 85 +35	God class — 45 unrelated methods, mutable static state, embedded secret, mixed concerns	Split by responsibility (domain service + repository + application service); remove static state
<code>app/Legacy/Services/CallRecordingGodService.php</code>	8, 10, 13, 21, 29, 37, 45, 53, 61, 69, 77, 85 +35	God class — 45 unrelated methods, mutable static state, embedded secret, mixed concerns	Split by responsibility (domain service + repository + application service); remove static state
<code>app/Legacy/Services/CallRoutingGodService.php</code>	8, 10, 13, 21, 29, 37, 45, 53, 61, 69, 77, 85 +35	God class — 45 unrelated methods, mutable static state, embedded secret, mixed concerns	Split by responsibility (domain service + repository + application service); remove static state
<code>app/Legacy/Services/CustomerProfileGodService.php</code>	8, 10, 13, 21, 29, 37, 45, 53, 61, 69, 77, 85 +35	God class — 45 unrelated methods, mutable static state, embedded secret, mixed concerns	Split by responsibility (domain service + repository + application service); remove static state
<code>app/Legacy/Services/DidInventoryGodService.php</code>	8, 10, 13, 21, 29, 37, 45, 53, 61, 69, 77, 85 +35	God class — 45 unrelated methods, mutable static state, embedded secret, mixed concerns	Split by responsibility (domain service + repository + application service); remove static state
<code>app/Legacy/Services/HistoricalReportsGodService.php</code>	8, 10, 13, 21, 29, 37, 45, 53, 61, 69, 77, 85 +35	God class — 45 unrelated methods, mutable static state, embedded secret, mixed concerns	Split by responsibility (domain service + repository + application service); remove static state
<code>app/Legacy/Services/LiveMonitoringGodService.php</code>	8, 10, 13, 21, 29, 37, 45, 53, 61, 69, 77, 85 +35	God class — 45 unrelated methods, mutable static state, embedded secret, mixed concerns	Split by responsibility (domain service + repository + application service); remove static state
<code>app/Legacy/Services/PromptLibraryGodService.php</code>	8, 10, 13, 21, 29, 37, 45, 53, 61, 69, 77, 85 +35	God class — 45 unrelated methods, mutable static state, embedded secret, mixed concerns	Split by responsibility (domain service + repository + application service); remove static state

<code>app/Legacy/Services/QueueManagementGodService.php</code>	8, 10, 13, 21, 29, 37, 45, 53, 61, 69, 77, 85 +35	God class — 45 unrelated methods, mutable static state, embedded secret, mixed concerns	Split by responsibility (domain service + repository + application service); remove static state
--	---	---	--

H8. Domain Boundary Violations MEDIUM

Benchmark: `Cross-domain access points = 4` → falls in the **Moderate** band (Good 0 · Moderate 1–5 · High Risk >5).

What to check: Code in one business area directly reading another area's tables/models.

Evidence:

`app/Http/Controllers/ReportsController.php:115-117` — the Reports domain reaches directly into the Call-Records and Queue operational tables:

```
return DB::table('ivr_call_records as c')
    ->leftJoin('ivr_operational_queues as q', 'q.id', '=', 'c.queue_id')
```

`app/Http/Controllers/Ivr/Concerns/LoadsIvrModuleData.php` and `app/Http/Controllers/Ivr/IvrHubController.php` each reference ≥5 distinct `ivr_*` tables to assemble a cross-module dashboard, with no owning context mediating access.

Why it matters here: Reporting and the Hub read the operational tables of Call Routing / Queue / Call Analytics directly, so those domains cannot rename a column or move to a separate store without silently breaking the reporting and hub screens. There is no published interface or anti-corruption layer between the read side and the domains it consumes.

Recommended approach:

1. Define bounded contexts (Queue, Call Routing, Analytics, Reporting) and forbid cross-context table access.
2. Expose read models / published interfaces (e.g. a `ReportingReadModel`) that each domain feeds, instead of Reports querying operational tables.
3. Wrap any unavoidable cross-domain read in an Anti-Corruption Layer translating the foreign schema.

3 files affected.

File	Line(s)	Issue	Action
<code>app/Http/Controllers/Ivr/Concerns/LoadsIvrModuleData.php</code>	82, 111, 140, 141, 185	Cross-domain access — one context queries another domain's operational tables directly	Introduce published read models / ACL; forbid direct cross-context table reads
<code>app/Http/Controllers/Ivr/IvrHubController.php</code>	68, 92, 132, 167, 204, 221, 222, 232, 258, 283, 284, 316 +2	Cross-domain access — one context queries another domain's operational tables directly	Introduce published read models / ACL; forbid direct cross-context table reads
<code>app/Http/Controllers/ReportsController.php</code>	61, 77, 98, 115, 116, 166, 167	Cross-domain access — one context queries another domain's operational tables directly	Introduce published read models / ACL; forbid direct cross-context table reads

H9. Shared Database Coupling MEDIUM

Benchmark: `Tables shared across domains = ~21%` (3 of 14 tables) → falls in the **Moderate** band (Good <10% · Moderate 10–30% · High Risk >30%).

What to check: Multiple domains reading/writing the same tables with no ownership.

Evidence:

`app/Http/Controllers/ReportsController.php:61,77,98` — the reporting screen reads `ivr_daily_trends`, `ivr_call_records` and `ivr_operational_queues`, which are populated by the operational IVR modules (e.g. the `*GodService` inserts into `ivr_*` tables at `QueueManagementGodService.php:18`). These three dashboard tables are shared write-by-one / read-by-another with no ownership boundary:

```
$base = DB::table('ivr_call_records') // ReportsController:77 – read side
...
return DB::table("ivr_queue_managements")->insertGetId(...) // GodService:18 – write side
```

Why it matters here: A schema change to the operational tables (indexing, column rename, partitioning) by the team that owns Queue/Call Routing will silently break the Reports and Hub screens, because ownership is implicit and enforced nowhere. This is the coupling that most blocks later extraction of any IVR module into its own service.

Recommended approach:

1. Assign each `ivr_*` table a single owning context; document ownership in the migration.
2. Feed reporting via an owned denormalized read table / materialized view instead of joining live operational tables.
3. Introduce internal APIs / events for cross-domain data needs and an ACL at each consumer.

14 files affected.

File	Line(s)	Issue	Action
<code>app/Http/Controllers/Ivr/Concerns/LoadsIvrModuleData.php</code>	82, 111, 140, 141, 185	Table read/written across multiple domains with no ownership boundary	Assign table ownership; serve cross-domain reads via owned read models or events
<code>app/Http/Controllers/Ivr/IvrHubController.php</code>	68, 92, 132, 167, 204, 221, 222, 232, 258, 283, 284, 316 +2	Table read/written across multiple domains with no ownership boundary	Assign table ownership; serve cross-domain reads via owned read models or events
<code>app/Http/Controllers/Ivr/QueueManagementDestroyController.php</code>	28	Table read/written across multiple domains with no ownership boundary	Assign table ownership; serve cross-domain reads via owned read models or events
<code>app/Http/Controllers/Ivr/QueueManagementExportController.php</code>	28	Table read/written across multiple domains with no ownership boundary	Assign table ownership; serve cross-domain reads via owned read models or events
<code>app/Http/Controllers/Ivr/QueueManagementImportController.php</code>	28	Table read/written across multiple domains with no ownership boundary	Assign table ownership; serve cross-domain reads via owned read models or events
<code>app/Http/Controllers/Ivr/QueueManagementIndexController.php</code>	28	Table read/written across multiple domains with no ownership boundary	Assign table ownership; serve cross-domain reads via owned read models or events
<code>app/Http/Controllers/Ivr/QueueManagementStoreController.php</code>	28	Table read/written across multiple domains with no ownership boundary	Assign table ownership; serve cross-domain reads via owned read models or events

<code>app/Http/Controllers/Ivr/QueueManagementSyncController.php</code>	28	Table read/written across multiple domains with no ownership boundary	Assign table ownership; serve cross-domain reads via owned read models or events
<code>app/Http/Controllers/Ivr/QueueManagementUpdateController.php</code>	28	Table read/written across multiple domains with no ownership boundary	Assign table ownership; serve cross-domain reads via owned read models or events
<code>app/Http/Controllers/ReportsController.php</code>	61, 77, 98, 115, 116, 166, 167	Table read/written across multiple domains with no ownership boundary	Assign table ownership; serve cross-domain reads via owned read models or events
<code>app/Legacy/Services/QueueManagementGodService.php</code>	18, 26, 34, 42, 50, 58, 66, 74, 82, 90, 98, 106 +33	Table read/written across multiple domains with no ownership boundary	Assign table ownership; serve cross-domain reads via owned read models or events
<code>app/Models/Ivr/QueueManagement.php</code>	13, 25, 31, 37, 43, 49, 55, 61, 67, 73, 79, 85 +24	Table read/written across multiple domains with no ownership boundary	Assign table ownership; serve cross-domain reads via owned read models or events
<code>app/Repositories/Legacy/QueueManagementRepository.php</code>	12, 21, 30, 39, 48, 57, 66, 75, 84, 93, 102, 111 +28	Table read/written across multiple domains with no ownership boundary	Assign table ownership; serve cross-domain reads via owned read models or events
<code>app/Support/IvrAccountContext.php</code>	62, 70	Table read/written across multiple domains with no ownership boundary	Assign table ownership; serve cross-domain reads via owned read models or events

H10. No Dependency Injection / Service-Locator Abuse (additional) CRITICAL

KPI definition: Services resolved via `new` instead of the DI container — Good 0 · Moderate 1–10 · High Risk >10. This hotspot directly targets the objective's **Dependency Injection** goal and is distinct from H1/H2 (it is about wiring, not location, of logic).

Benchmark: Manual instantiations = 80 controllers do `new *GodService()` ;
`AppServiceProvider` registers 0 bindings → **High Risk** band.

What to check: Concrete classes created with `new` (or fetched from a static locator) instead of injected; empty container configuration.

Evidence:

`app/Http/Controllers/Ivr/QueueManagementStoreController.php:24, 47` — services are hand-instantiated inside every action:

```
$service = new QueueManagementGodService();
```

`app/Providers/AppServiceProvider.php:27, 42` — the container is empty of domain bindings and disables mass-assignment protection globally:

```
public function register(): void
{
```

```
Model::unguard(); // no interface-implementation bindings anywhere
}
```

No IVR controller declares a `__construct` (grep returns 0), confirming zero constructor injection.

Why it matters here: Because services and (would-be) repositories are `new`-ed inside handlers, nothing can be substituted — no test double, no decorator for caching/logging, no swap of the SQL repository for an API-backed one. `Model::unguard()` compounds this by making every model mass-assignable, so the missing DI boundary is also a security surface. Introducing bounded contexts is impossible until construction is inverted.

Recommended approach:

1. Bind each service/repository interface to its implementation in `AppServiceProvider::register()`.
2. Replace every `new *GodService()` with a constructor-injected dependency typed to the interface.
3. Remove `Model::unguard()` and add per-model `$fillable`, closing the mass-assignment hole opened by the missing boundary.

80 files affected.

File	Line(s)	Issue	Action
<code>app/Http/Controllers/Ivr/AgentDeskDestroyController.php</code>	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
<code>app/Http/Controllers/Ivr/AgentDeskExportController.php</code>	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
<code>app/Http/Controllers/Ivr/AgentDeskImportController.php</code>	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
<code>app/Http/Controllers/Ivr/AgentDeskIndexController.php</code>	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
<code>app/Http/Controllers/Ivr/AgentDeskStoreController.php</code>	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
<code>app/Http/Controllers/Ivr/AgentDeskSyncController.php</code>	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
<code>app/Http/Controllers/Ivr/AgentDeskUpdateController.php</code>	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
<code>app/Http/Controllers/Ivr/BusinessHoursDestroyController.php</code>	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
<code>app/Http/Controllers/Ivr/BusinessHoursExportController.php</code>	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor

[illegible]

[illegible]

app/Http/Controllers/Ivr/CallRoutingIndexController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/CallRoutingStoreController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/CallRoutingSyncController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/CallRoutingUpdateController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/CustomerProfileIndexController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/CustomerProfileStoreController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/CustomerProfileUpdateController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/DidInventoryDestroyController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/DidInventoryExportController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/DidInventoryImportController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/DidInventoryIndexController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/DidInventoryStoreController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/DidInventorySyncController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/DidInventoryUpdateController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/HistoricalReportsDestroyController.php	25, 49, 62, 75, 88, 101, 114,	Service created with `new` instead of injected — no	Bind an interface in AppServiceProvider and inject it via the constructor

	127, 140, 153, 166, 179 +44	DI, untestable, unsubstitutable	
app/Http/Controllers/Ivr/HistoricalReportsExportController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/HistoricalReportsImportController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/HistoricalReportsIndexController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/HistoricalReportsStoreController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/HistoricalReportsSyncController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/HistoricalReportsUpdateController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/LiveMonitoringDestroyController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/LiveMonitoringExportController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/LiveMonitoringImportController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/LiveMonitoringIndexController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/LiveMonitoringStoreController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/LiveMonitoringSyncController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/LiveMonitoringUpdateController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/PromptLibraryDestroyController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor

app/Http/Controllers/Ivr/PromptLibraryExportController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/PromptLibraryImportController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/PromptLibraryIndexController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/PromptLibraryStoreController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/PromptLibrarySyncController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/PromptLibraryUpdateController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/QueueManagementDestroyController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/QueueManagementExportController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/QueueManagementImportController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/QueueManagementIndexController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/QueueManagementStoreController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/QueueManagementSyncController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor
app/Http/Controllers/Ivr/QueueManagementUpdateController.php	25, 49, 62, 75, 88, 101, 114, 127, 140, 153, 166, 179 +44	Service created with `new` instead of injected — no DI, untestable, unsubstitutable	Bind an interface in AppServiceProvider and inject it via the constructor

F1. Business Logic in Components MEDIUM

Benchmark: Avg component LOC = 139 , but 134 of 522 Page components (26%) exceed 300 LOC → worst KPI falls in the **Moderate** band (Good <150 avg · Moderate 150–300 · High Risk >300).

What to check: Validation/transformation/workflow logic living inside React components instead of hooks/services.

Evidence:

`resources/js/Pages/Ivr/CallFlow/Sync.tsx:22-27` — client-side validation that admits it duplicates the PHP controller, plus hard-coded tenancy, inside the view:

```
const validateClientSide = (payload: Record<string, unknown>) => {
  // duplicate validation - also exists in PHP controller
  if (!payload.name) return 'Name required'
  return null
}
```

The same 392-LOC `LegacyPass2_*.tsx` component shape (with inline logic + `useEffect` fetch) is repeated hundreds of times across the IVR pages.

Why it matters here: Validation and transformation rules are copied into components *and* the PHP controllers, so a rule change (e.g. a new required field) must be edited in both layers or the two silently diverge. With 134 oversized page components, that divergence surface is large and grows every time a module is cloned.

Recommended approach:

1. Extract validation/transform logic into shared hooks (`useCallFlowForm`) or a typed client, and have components call it.
2. Derive the client rule from a single shared schema (e.g. Zod) that also documents the server contract, eliminating the duplicated `validateClientSide`.
3. Split the 392-LOC `LegacyPass2_*` template into a presentational component + a data hook.

374 files affected.

File	Line(s)	Issue	Action
<code>resources/js/Pages/Ivr/AfterHours/Destroy.tsx</code>	22	Validation/business logic embedded in the view component (duplicates server rules)	Move logic into a shared hook/schema; keep the component presentational
<code>resources/js/Pages/Ivr/AfterHours/Export.tsx</code>	22	Validation/business logic embedded in the view component (duplicates server rules)	Move logic into a shared hook/schema; keep the component presentational
<code>resources/js/Pages/Ivr/AfterHours/Import.tsx</code>	22	Validation/business logic embedded in the view component (duplicates server rules)	Move logic into a shared hook/schema; keep the component presentational
<code>resources/js/Pages/Ivr/AfterHours/Index.tsx</code>	22	Validation/business logic embedded in the view component (duplicates server rules)	Move logic into a shared hook/schema; keep the component presentational
<code>resources/js/Pages/Ivr/AfterHours/Monitor.tsx</code>	22	Validation/business logic embedded in the view component (duplicates server rules)	Move logic into a shared hook/schema; keep the component presentational
<code>resources/js/Pages/Ivr/AfterHours/Store.tsx</code>	22	Validation/business logic embedded in the view component (duplicates server rules)	Move logic into a shared hook/schema; keep the component presentational
<code>resources/js/Pages/Ivr/AfterHours/Sync.tsx</code>	22	Validation/business logic embedded in the view component (duplicates server rules)	Move logic into a shared hook/schema; keep the component presentational
<code>resources/js/Pages/Ivr/AfterHours/Update.tsx</code>	22	Validation/business logic embedded in the view component (duplicates server rules)	Move logic into a shared hook/schema; keep the component presentational

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

<code>resources/js/Pages/Ivr/WhisperCoach/Index.tsx</code>	22	Validation/business logic embedded in the view component (duplicates server rules)	Move logic into a shared hook/schema; keep the component presentational
<code>resources/js/Pages/Ivr/WhisperCoach/Monitor.tsx</code>	22	Validation/business logic embedded in the view component (duplicates server rules)	Move logic into a shared hook/schema; keep the component presentational
<code>resources/js/Pages/Ivr/WhisperCoach/Store.tsx</code>	22	Validation/business logic embedded in the view component (duplicates server rules)	Move logic into a shared hook/schema; keep the component presentational
<code>resources/js/Pages/Ivr/WhisperCoach/Sync.tsx</code>	22	Validation/business logic embedded in the view component (duplicates server rules)	Move logic into a shared hook/schema; keep the component presentational
<code>resources/js/Pages/Ivr/WhisperCoach/Update.tsx</code>	22	Validation/business logic embedded in the view component (duplicates server rules)	Move logic into a shared hook/schema; keep the component presentational

F2. Missing Frontend Service/Data Layer HIGH

Benchmark: `Components with inline API calls = 374` → falls in the **High Risk** band (Good <10 · Moderate 10–20 · High Risk >20).

What to check: `fetch` / `axios` / `router.*` calls and API URLs hard-coded inline in components instead of a shared client/service layer.

Evidence:

`resources/js/Pages/Ivr/CallFlow/Sync.tsx:13-20` — a hard-coded URL fetched on an interval with no cleanup and no shared client:

```
useEffect(() => {
  // missing cleanup - interval leak pattern
  const id = setInterval(() => {
    fetch('/ivr-legacy/call-flow/sync?q=' + search)
      .then(r => r.json())
      .then(d => setLocalRows(d.data ?? localRows))
      .catch(() => {})
  }, 5000)
}, [search])
```

`resources/js/legacy/class/AfterHoursClassWidget0.jsx:5-6` — the same inline-fetch pattern in a class component:

```
componentDidMount() {
  fetch('/ivr-legacy/after-hours/index').then(r => r.json()).then(d =>
    this.setState({ rows: d.data || [] }))
}
```

374 components hard-code their own endpoints and error handling.

Why it matters here: With no shared data layer, cross-cutting concerns — auth headers, base URL, retry, error toasts, cancellation — cannot be applied in one place, and the `/ivr-legacy/...` URL scheme is baked into 374 files. Changing an endpoint path

or adding a CSRF header is a 374-file edit, and the missing `clearInterval` in `Sync.tsx` leaks a timer on every remount.

Recommended approach:

1. Introduce a single typed API client / service module (`services/ivrClient.ts`) that owns base URL, headers and error handling.
2. Wrap data access in hooks (`useCallFlowSync`) that call the client and handle cleanup/cancellation.
3. Replace the 374 inline `fetch(...)` calls with the hook/client; codemod the URL literals into the client.

760 files affected.

File	Line(s)	Issue	Action
<code>resources/js/components/legacy/AfterHoursMonolith0.tsx</code>	10	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/components/legacy/AfterHoursMonolith1.tsx</code>	10	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/components/legacy/AfterHoursMonolith2.tsx</code>	10	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/components/legacy/AfterHoursMonolith3.tsx</code>	10	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/components/legacy/AfterHoursMonolith4.tsx</code>	10	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/components/legacy/AgentDeskMonolith0.tsx</code>	10	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/components/legacy/AgentDeskMonolith1.tsx</code>	10	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/components/legacy/AgentDeskMonolith2.tsx</code>	10	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/components/legacy/AgentDeskMonolith3.tsx</code>	10	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/components/legacy/AgentDeskMonolith4.tsx</code>	10	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/components/legacy/ApiIntegrationMonolith0.tsx</code>	10	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/components/legacy/ApiIntegrationMonolith1.tsx</code>	10	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/components/legacy/ApiIntegrationMonolith2.tsx</code>	10	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/components/legacy/ApiIntegrationMonolith3.tsx</code>	10	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

<code>resources/js/Pages/Organizations/Edit.tsx</code>	57, 94	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/Pages/Organizations/Index.tsx</code>	34	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/Pages/Reports/Index.tsx</code>	82	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/Pages/Users/Edit.tsx</code>	47, 90	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components
<code>resources/js/Pages/Users/Index.tsx</code>	35	Inline API/data-access call with a hard-coded URL in the component (no shared client)	Move the call into a typed API client/hook; remove hard-coded URLs from components

F3. God / Oversized Components MEDIUM

Benchmark: `Components >400 LOC = 1` (`Pages/Ivr/Hub/Index.tsx` = 479 LOC) → falls in the **Moderate** band (Good 0 · Moderate 1–3 · High Risk >3); 134 further components sit in the 300–400 band just under the threshold.

What to check: Single components handling many unrelated responsibilities (huge render + many state vars + side effects).

Evidence:

`resources/js/Pages/Ivr/Hub/Index.tsx` (479 LOC) is the single largest component — an IVR hub that renders and wires many modules in one file. It is the outlier above a dense cluster of 134 components in the 300–400-LOC band (e.g. the 392-LOC `LegacyPass2_*.tsx` template), so the oversized-component problem is broad even though only one file breaches 400.

Why it matters here: The Hub concentrates the cross-module dashboard in one 479-line component, so every IVR module change risks the hub render, and the 134 near-400-LOC page templates mean the codebase is one refactor away from many more god components. As modules are added by cloning `LegacyPass2_*`, several will cross the 400-line line.

Recommended approach:

1. Decompose `Hub/Index.tsx` into per-module widget components fed by hooks, leaving the page as a thin layout.
2. Extract the shared 392-LOC `LegacyPass2_*` render into a reusable `<ModuleTable>` + data hook so cloning stops adding 392 lines each time.

1 file affected.

File	Line(s)	Issue	Action
<code>resources/js/Pages/Ivr/Hub/Index.tsx</code>	—	Oversized component concentrating many module responsibilities in one file	Decompose into per-module widgets fed by hooks; keep the page as a thin layout

F5. Legacy / Inconsistent Component Patterns HIGH

Benchmark: `Legacy class components = 147` → falls in the **High Risk** band (Good 0 · Moderate 1–10 · High Risk >10). Mixed with 769 function components and no error boundaries.

What to check: Mixed paradigms (class + function components), deprecated lifecycle APIs, missing error boundaries, no shared conventions.

Evidence:

`resources/js/legacy/class/AfterHoursClassWidget0.jsx:3-6` — a React class component using `componentDidMount` + inline fetch, one of 147 such widgets alongside a function-component-based Pages tree:

```
export default class AfterHoursClassWidget0 extends React.Component {
  state = { count: 0, rows: [] }
  componentDidMount() {
    fetch('/ivr-legacy/after-hours/index').then(r => r.json()).then(d =>
this.setState({ rows: d.data || [] }))
  }
}
```

The project runs React 19 (function components + hooks) yet carries 147 class components in `resources/js/legacy/class/`, with no error boundaries anywhere in the tree.

Why it matters here: Two component paradigms coexist with no shared convention, so contributors must know both, and the class widgets cannot use the hooks/typed-client introduced for the function components — freezing 147 files out of any modernization. The absence of error boundaries means a single failed legacy fetch can blank a screen.

Recommended approach:

1. Codemod the 147 `*ClassWidget*.jsx` to function components with hooks and the shared data client (F2).
2. Add a top-level error boundary and a lint rule (`react/prefer-stateless-function` / ban `React.Component`) to prevent regressions.
3. Standardize on one component convention (function + hooks + TS) and migrate `.jsx` legacy widgets to `.tsx`.

147 files affected.

File	Line(s)	Issue	Action
<code>resources/js/legacy/class/AfterHoursClassWidget0.jsx</code>	3	Legacy React class component mixed into a hooks-based codebase; no error boundary	Migrate to a function component with hooks and the shared data client
<code>resources/js/legacy/class/AfterHoursClassWidget1.jsx</code>	3	Legacy React class component mixed into a hooks-based codebase; no error boundary	Migrate to a function component with hooks and the shared data client
<code>resources/js/legacy/class/AfterHoursClassWidget2.jsx</code>	3	Legacy React class component mixed into a hooks-based codebase; no error boundary	Migrate to a function component with hooks and the shared data client
<code>resources/js/legacy/class/AfterHoursClassWidget3.jsx</code>	3	Legacy React class component mixed into a hooks-based codebase; no error boundary	Migrate to a function component with hooks and the shared data client
<code>resources/js/legacy/class/AgentDeskClassWidget1.jsx</code>	3	Legacy React class component mixed into a hooks-based codebase; no error boundary	Migrate to a function component with hooks and the shared data client
<code>resources/js/legacy/class/AgentDeskClassWidget2.jsx</code>	3	Legacy React class component mixed into a hooks-based codebase; no error boundary	Migrate to a function component with hooks and the shared data client
<code>resources/js/legacy/class/AgentDeskClassWidget3.jsx</code>	3	Legacy React class component mixed into a hooks-based codebase; no error boundary	Migrate to a function component with hooks and the shared data client
<code>resources/js/legacy/class/ApiIntegrationClassWidget0.jsx</code>	3	Legacy React class component mixed into a hooks-based codebase; no error boundary	Migrate to a function component with hooks and the shared data client
<code>resources/js/legacy/class/ApiIntegrationClassWidget1.jsx</code>	3	Legacy React class component mixed into a hooks-based codebase; no error boundary	Migrate to a function component with hooks and the shared data client

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

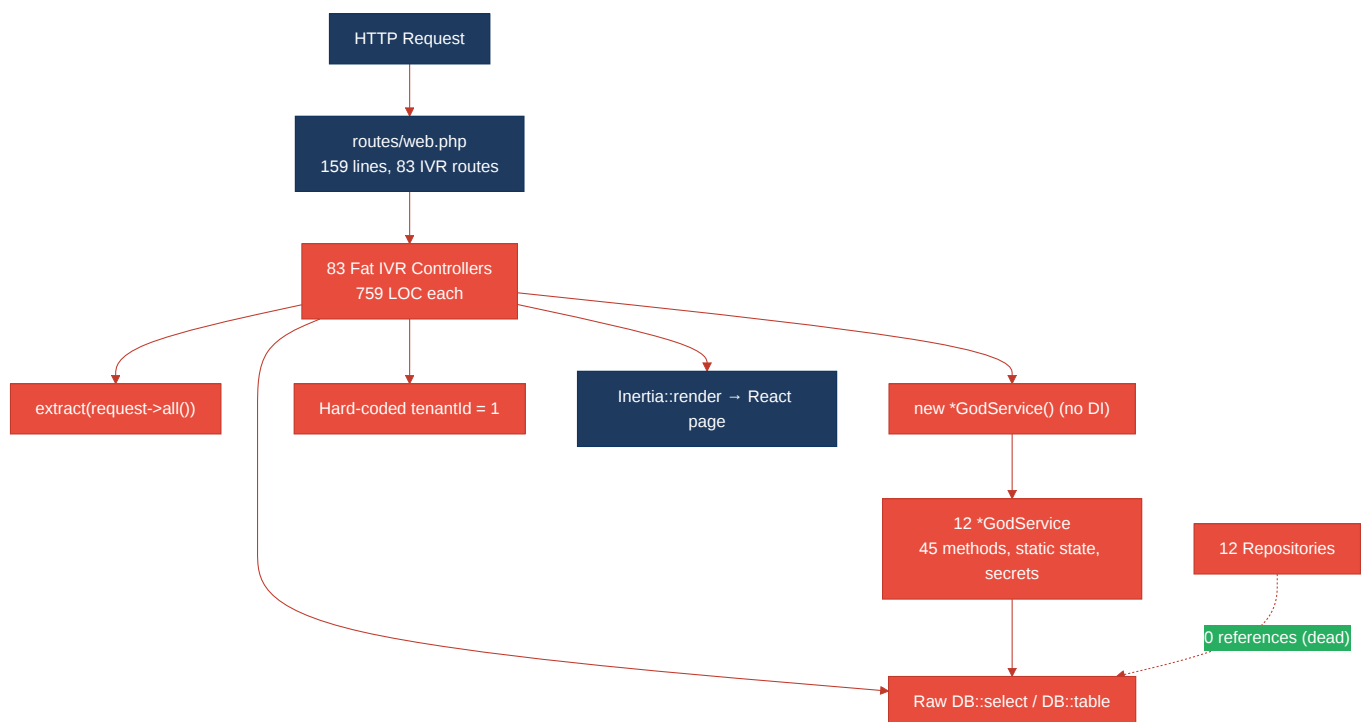
[illegible]

<code>resources/js/legacy/class/WebhookDispatcherClassWidget0.jsx</code>	3	Legacy React class component mixed into a hooks-based codebase; no error boundary	Migrate to a function component with hooks and the shared data client
<code>resources/js/legacy/class/WebhookDispatcherClassWidget2.jsx</code>	3	Legacy React class component mixed into a hooks-based codebase; no error boundary	Migrate to a function component with hooks and the shared data client
<code>resources/js/legacy/class/WebhookDispatcherClassWidget4.jsx</code>	3	Legacy React class component mixed into a hooks-based codebase; no error boundary	Migrate to a function component with hooks and the shared data client
<code>resources/js/legacy/class/WhisperCoachClassWidget0.jsx</code>	3	Legacy React class component mixed into a hooks-based codebase; no error boundary	Migrate to a function component with hooks and the shared data client
<code>resources/js/legacy/class/WhisperCoachClassWidget2.jsx</code>	3	Legacy React class component mixed into a hooks-based codebase; no error boundary	Migrate to a function component with hooks and the shared data client

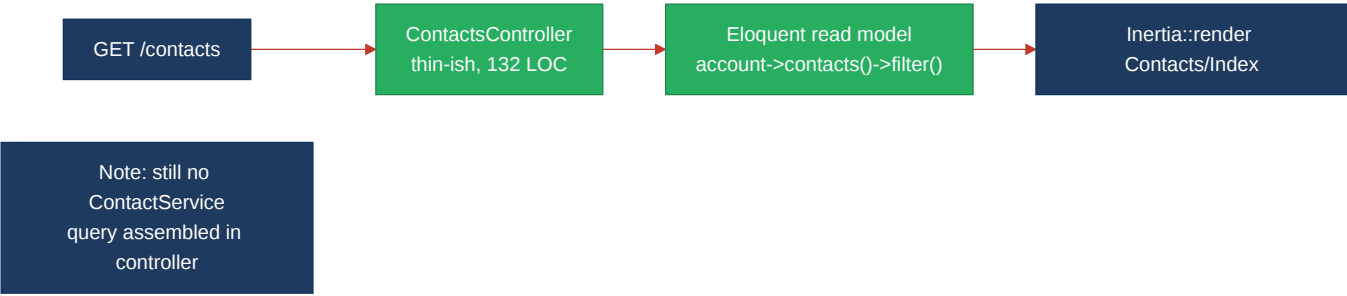
Not observed (rated Good): H4 Circular Dependencies — dependency flow is linear Controller → (new)Service → DB with no import cycles between modules/packages; F4 Prop Drilling / Global State Abuse — components self-fetch into local `useState` (max depth ≤2), and no oversized global store/context was found.

1.3 Diagrams

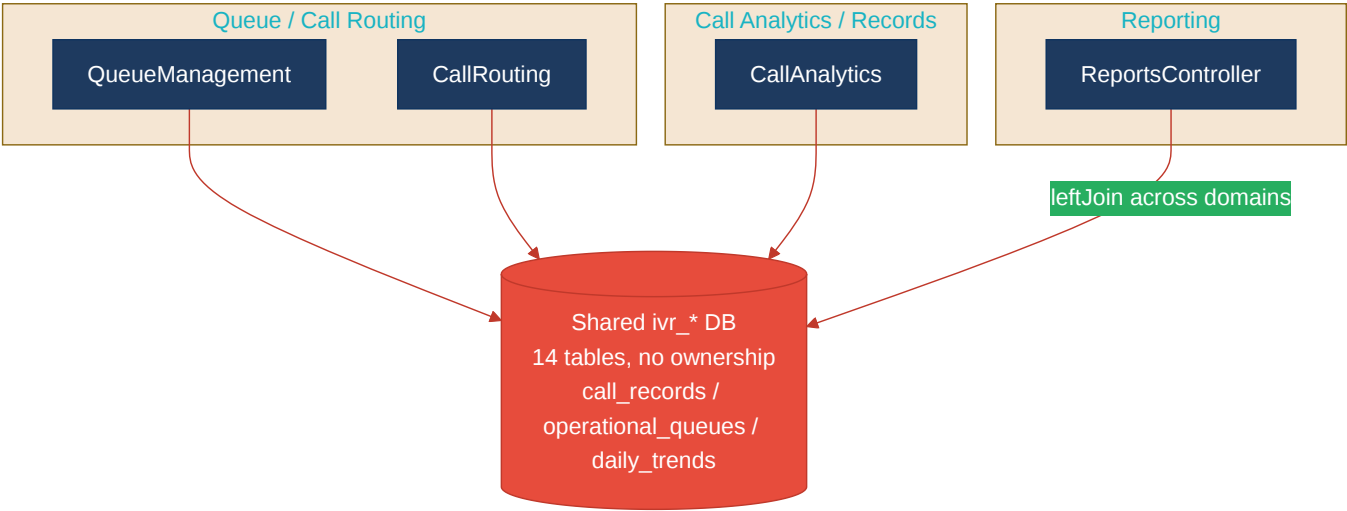
Current-state architecture (as-is)



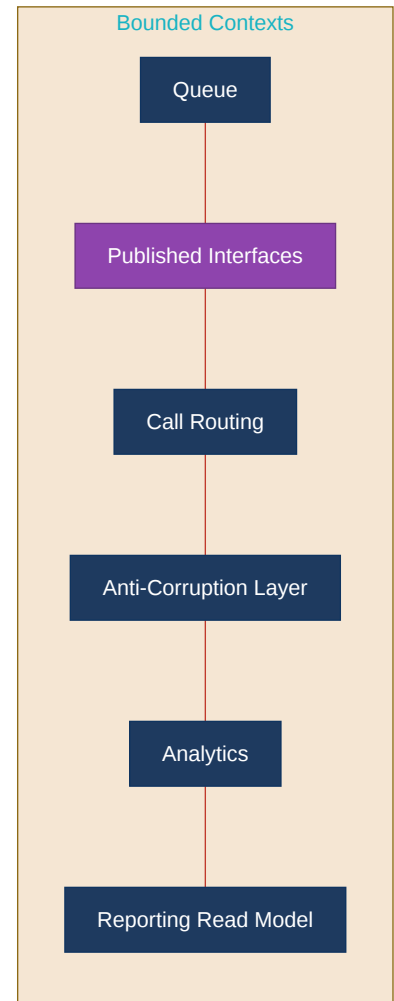
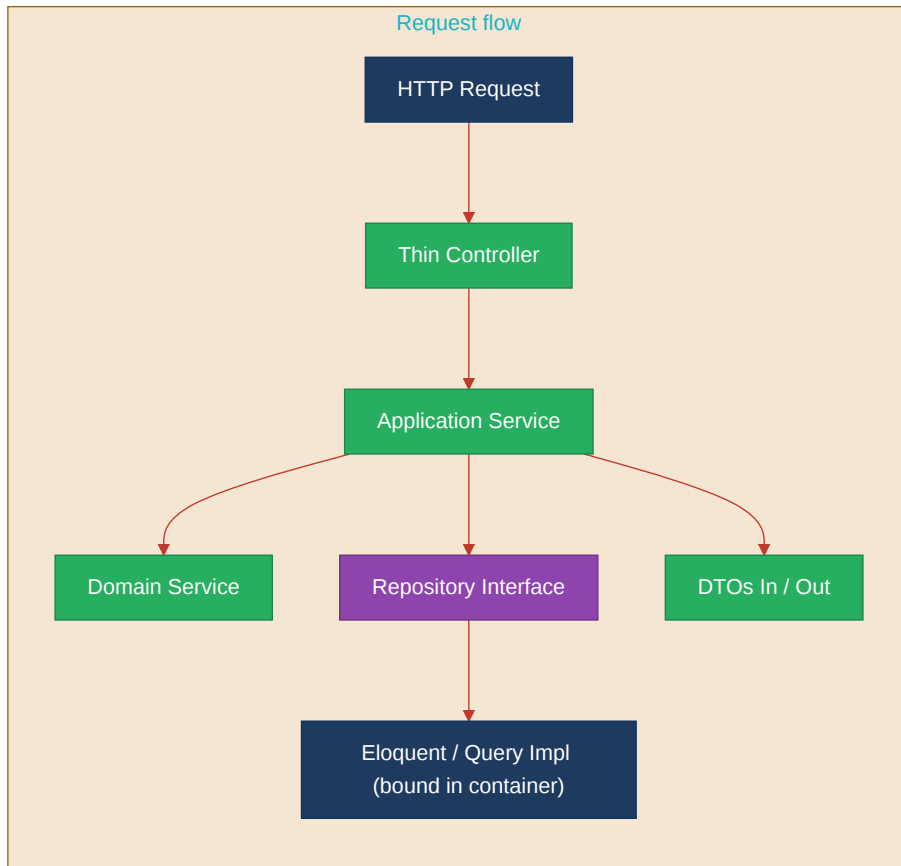
Clean reference path (closest existing "good" example)



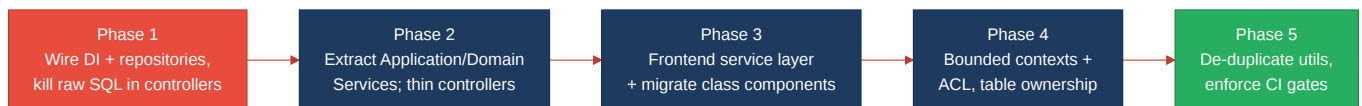
Domain boundary map (business domains vs. shared data)



Target architecture (proposed)



Improvement roadmap



1.4 Actions Required

Hotspot	Action	Rating	Priority
H1 Fat Controllers	Reduce the 83 IVR controllers to validate-and-delegate; extract logic into Application Services	HIGH RISK	CRITICAL
H2 Missing Service Layer	Introduce per-capability Application Services; move query/workflow assembly out of controllers	HIGH RISK	CRITICAL
H6 Direct SQL in Controllers	Move all <code>DB::</code> queries out of controllers into bound-parameter repositories; add CI grep gate	HIGH RISK	CRITICAL
H10 No Dependency Injection	Bind interfaces in <code>AppServiceProvider</code> , inject services, remove <code>new *GodService()</code> and <code>Model::unguard()</code>	HIGH RISK	CRITICAL

H3 Missing Repository Pattern	Give repositories interfaces + bound queries; route all persistence through them (kill the dead tier)	HIGH RISK	HIGH
H5 Shared Utility Abuse	Replace 5 <code>LegacyIvr*</code> helpers with domain services; collapse 8 duplicate formatter modules into one	HIGH RISK	HIGH
H7 God Classes	Split each <code>*GodService</code> into domain service + repository + application service; remove static state/secret	HIGH RISK	HIGH
F2 Missing Frontend Service Layer	Introduce a typed API client + hooks; replace 374 inline <code>fetch()</code> /hard-coded URLs	HIGH RISK	HIGH
F5 Legacy Component Patterns	Codemod 147 class widgets to hooks; add error boundaries and a ban-class lint rule	HIGH RISK	HIGH
H8 Domain Boundary Violations	Introduce published read models / ACL; forbid cross-context table reads in Reports & Hub	MODERATE	MEDIUM
H9 Shared Database Coupling	Assign per-table ownership; serve reporting via an owned read model instead of live joins	MODERATE	MEDIUM
F1 Business Logic in Components	Move validation/transform into shared hooks/schema; split 300–400-LOC page templates	MODERATE	MEDIUM
F3 God / Oversized Components	Decompose <code>Hub/Index.tsx</code> (479 LOC) and the shared 392-LOC template into widgets + hooks	MODERATE	MEDIUM

1.5 Expected Outcomes

- **Testable, reusable business logic** — with Application/Domain Services and constructor DI in place, workflows can be unit-tested with mocked repositories and reused from HTTP, CLI and queued jobs instead of being copy-pasted across 83 controllers.
- **Schema and persistence freedom** — funnelling every `DB::` call through parameter-bound repositories removes the injection holes and lets the team rename/repartition `ivr_*` tables without editing dozens of hand-written query strings.
- **Independent, extractable domains** — bounded contexts with published interfaces and an anti-corruption layer let Queue, Call Routing, Analytics and Reporting evolve (and eventually extract) without silently breaking each other through shared tables.
- **A consistent, maintainable frontend** — a single typed API client plus hooks and function components collapses 374 inline fetches and 147 class widgets into one convention, so endpoint/auth changes are one-file edits and screens stop leaking timers and blanking on errors.
- **Sharply lower change amplification** — de-duplicating the 5 helpers and 8 formatter modules and enforcing CI gates (no `DB::` in controllers, no `React.Component`, duplication check) means a single rule change stops rippling through dozens of near-identical files.